



Lecture 6: Building a Production RAG Project

End-to-End System Design for Your Portfolio

Part of the RAG Mastery Series



What You Will Learn Today

1. How to design a production RAG system from scratch
 2. Complete project: "AI Research Assistant"
 3. Architecture decisions and trade-offs
 4. Implementation step-by-step
 5. Evaluation and monitoring
 6. Deployment considerations
-

1. Project: AI Research Assistant

What We're Building

A RAG system that lets users upload research papers (PDFs) and ask questions about them. Think of it as "ChatGPT for your research papers."

User Flow:

1. User uploads PDFs (research papers)
2. System processes and indexes them
3. User asks questions in natural language
4. System retrieves relevant chunks and generates answers
5. System provides citations (which paper, which page)

Why This Project?

- ✓ Great for your portfolio (shows end-to-end RAG skills)
- ✓ Solves a real problem (researchers need this)

- ✓ Demonstrates multiple RAG techniques
 - ✓ Can be extended (multimodal, agentic, etc.)
 - ✓ Free to build (uses open-source tools)
-

2. Architecture Design

System Architecture

AI RESEARCH ASSISTANT

FRONTEND (Streamlit)

Upload PDFs | Ask Questions | View Sources | Chat History

HTTP

BACKEND (FastAPI)

Upload
Handler

Query
Handler

Evaluate
Endpoint

Feedback
Endpoint

RAG PIPELINE

Document
Processor

Retrieval
Engine

Generation
(LLM)

- Parse
- Chunk
- Embed
- Store

- Hybrid
- Rerank
- Filter

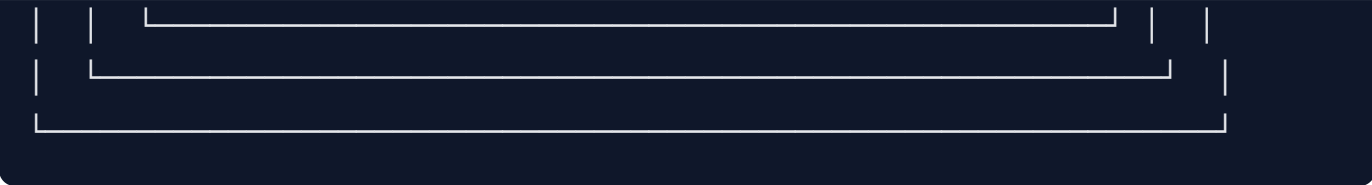
- Prompt Builder
- Context Window Management
- Citation

DATA LAYER

Vector DB
(Qdrant/
TurboVec)

Metadata
Store
(SQLite)

Document Store
(Local/S3)



3. Implementation — Step by Step

Step 1: Project Setup

```
# Create project
mkdir ai-research-assistant
cd ai-research-assistant
python -m venv venv
source venv/Scripts/activate # Windows

# Install dependencies
pip install fastapi uvicorn streamlit requests
pip install langchain langchain-community
pip install sentence-transformers
pip install qdrant-client
pip install pymupdf # PDF parsing
pip install python-multipart # File upload
pip install tiktoken # Token counting
```

Step 2: Document Processing

```

# document_processor.py
import fitz # PyMuPDF
from langchain.text_splitter import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
import hashlib

class DocumentProcessor:
    def __init__(self):
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=512,
            chunk_overlap=64,
            separators=["\n\n", "\n", ". ", " ", ""]
        )

    def parse_pdf(self, file_path: str) -> dict:
        """Extract text and metadata from PDF."""
        doc = fitz.open(file_path)
        pages = []

        for i, page in enumerate(doc):
            pages.append({
                "page_number": i + 1,
                "text": page.get_text()
            })

        full_text = "\n".join([p["text"] for p in pages])

        return {
            "filename": file_path.split("/")[-1],
            "total_pages": len(doc),
            "text": full_text,
            "pages": pages,
            "doc_id": hashlib.md5(full_text.encode()).hexdigest()[:12]
        }

    def chunk_document(self, doc: dict) -> list[dict]:

```

```

"""Split document into chunks with metadata."""
chunks = []

# Split by pages first to maintain page references
for page in doc["pages"]:
    page_chunks = self.splitter.split_text(page["text"])

    for chunk_text in page_chunks:
        if len(chunk_text.strip()) > 50: # Skip tiny chunks
            chunks.append({
                "text": chunk_text,
                "doc_id": doc["doc_id"],
                "filename": doc["filename"],
                "page_number": page["page_number"],
                "chunk_id": f"
{doc['doc_id']}_p{page['page_number']}_{'len(chunks)}"
                })

    return chunks

def embed_chunks(self, chunks: list[dict]) -> list[dict]:
    """Generate embeddings for all chunks."""
    texts = [c["text"] for c in chunks]
    embeddings = self.encoder.encode(texts, show_progress_bar=True)

    for i, chunk in enumerate(chunks):
        chunk["embedding"] = embeddings[i].tolist()

    return chunks

def process(self, file_path: str) -> list[dict]:
    """Full pipeline: parse → chunk → embed."""
    doc = self.parse_pdf(file_path)
    chunks = self.chunk_document(doc)
    chunks = self.embed_chunks(chunks)
    return chunks

```

Step 3: Vector Storage

```

# vector_store.py
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import uuid

class VectorStore:
    def __init__(self, collection_name="research_papers"):
        self.client = QdrantClient("localhost", port=6333)
        self.collection_name = collection_name
        self._ensure_collection()

    def _ensure_collection(self):
        """Create collection if it doesn't exist."""
        try:
            self.client.get_collection(self.collection_name)
        except:
            self.client.create_collection(
                collection_name=self.collection_name,
                vectors_config=VectorParams(
                    size=384, # all-MiniLM-L6-v2 dimensions
                    distance=Distance.COSINE
                )
            )

    def add_chunks(self, chunks: list[dict]):
        """Store chunks in vector database."""
        points = []
        for chunk in chunks:
            points.append(PointStruct(
                id=str(uuid.uuid4()),
                vector=chunk["embedding"],
                payload={
                    "text": chunk["text"],
                    "doc_id": chunk["doc_id"],
                    "filename": chunk["filename"],
                    "page_number": chunk["page_number"],
                    "chunk_id": chunk["chunk_id"]
                }
            ))

```

```

        }
    ))

    self.client.upsert(
        collection_name=self.collection_name,
        points=points
    )
    print(f"Stored {len(points)} chunks")

def search(self, query_embedding: list, k=5, doc_id=None):
    """Search for relevant chunks."""
    filter_condition = None
    if doc_id:
        filter_condition = {
            "must": [
                {"key": "doc_id", "match": {"value": doc_id}}
            ]
        }

    results = self.client.search(
        collection_name=self.collection_name,
        query_vector=query_embedding,
        query_filter=filter_condition,
        limit=k,
        with_payload=True
    )

    return [
        {
            "score": r.score,
            "text": r.payload["text"],
            "filename": r.payload["filename"],
            "page_number": r.payload["page_number"]
        }
        for r in results
    ]

```

Step 4: RAG Pipeline

```

# rag_pipeline.py
from document_processor import DocumentProcessor
from vector_store import VectorStore
from sentence_transformers import SentenceTransformer

class RAGPipeline:
    def __init__(self):
        self.processor = DocumentProcessor()
        self.vector_store = VectorStore()
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')

    def index_document(self, file_path: str):
        """Process and index a PDF document."""
        chunks = self.processor.process(file_path)
        self.vector_store.add_chunks(chunks)
        return len(chunks)

    def query(self, question: str, k: int = 5):
        """Answer a question using RAG."""
        # Step 1: Encode query
        query_embedding = self.encoder.encode([question])[0].tolist()

        # Step 2: Retrieve relevant chunks
        results = self.vector_store.search(query_embedding, k=k)

        # Step 3: Build context
        context = ""
        for i, r in enumerate(results):
            context += f"[Source {i+1}: {r['filename']}, page {r['page_number']}]\\n"
            context += r["text"] + "\\n\\n"

        # Step 4: Build prompt
        prompt = self._build_prompt(question, context)

        # Step 5: Generate answer (using any LLM)
        # For this example, we return the prompt + sources

```

```
    return {
        "question": question,
        "prompt": prompt,
        "sources": results
    }
```

```
def _build_prompt(self, question: str, context: str) -> str:
    return f"""You are a research assistant. Use the following
context to answer the question accurately.
Cite sources using [Source N] notation. If the context doesn't contain
enough information, say so.
```

```
Context:
{context}
```

```
Question: {question}
```

```
Provide a clear, accurate answer with citations:"""
```

Step 5: FastAPI Backend

```

# main.py
from fastapi import FastAPI, UploadFile, File, HTTPException
from pydantic import BaseModel
import tempfile
import os
from rag_pipeline import RAGPipeline

app = FastAPI(title="AI Research Assistant")
rag = RAGPipeline()

class QueryRequest(BaseModel):
    question: str
    k: int = 5

class QueryResponse(BaseModel):
    question: str
    answer: str
    sources: list

@app.post("/upload")
async def upload_pdf(file: UploadFile = File(...)):
    """Upload and index a PDF."""
    if not file.filename.endswith(".pdf"):
        raise HTTPException(400, "Only PDF files allowed")

    # Save to temp file
    with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
        tmp.write(await file.read())
        tmp_path = tmp.name

    try:
        num_chunks = rag.index_document(tmp_path)
        return {"status": "success", "chunks_indexed": num_chunks}
    finally:
        os.unlink(tmp_path)

```

```
@app.post("/query", response_model=QueryResponse)
async def query(request: QueryRequest):
    """Ask a question."""
    result = rag.query(request.question, request.k)
    return QueryResponse(
        question=result["question"],
        answer=result["prompt"], # In production, send to LLM here
        sources=result["sources"]
    )

@app.get("/health")
async def health():
    return {"status": "healthy"}
```

Step 6: Streamlit Frontend

```

# app.py
import streamlit as st
import requests

st.set_page_config(page_title="AI Research Assistant", layout="wide")
st.title("🔬 AI Research Assistant")
st.markdown("Upload research papers and ask questions!")

BACKEND_URL = "http://localhost:8000"

# Sidebar: File Upload
st.sidebar.header("📄 Upload Papers")
uploaded_files = st.sidebar.file_uploader(
    "Upload PDFs", type="pdf", accept_multiple_files=True
)

if uploaded_files:
    for file in uploaded_files:
        if st.sidebar.button(f"Index: {file.name}"):
            files = {"file": (file.name, file.getvalue(),
"application/pdf")}
            response = requests.post(f"{BACKEND_URL}/upload",
files=files)
            if response.ok:
                st.sidebar.success(f"✅ Indexed! {response.json()
['chunks_indexed']} chunks")
            else:
                st.sidebar.error("❌ Upload failed")

# Main: Chat
st.header("💬 Ask Questions")

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):

```

```

        st.write(msg["content"])
    if "sources" in msg:
        with st.expander("📖 Sources"):
            for s in msg["sources"]:
                st.write(f"- **{s['filename']}** (page {s['page_number']}, score: {s['score']:.3f})")
                st.caption(s['text'][:200] + "...")

if prompt := st.chat_input("Ask a question about your papers..."):
    st.session_state.messages.append({"role": "user", "content": prompt})

    response = requests.post(
        f"{BACKEND_URL}/query",
        json={"question": prompt, "k": 5}
    )

    if response.ok:
        data = response.json()
        st.session_state.messages.append({
            "role": "assistant",
            "content": data["answer"],
            "sources": data["sources"]
        })
    else:
        st.error("Query failed")

```

4. Running the Project

```
# Terminal 1: Start Qdrant (Docker)
docker run -p 6333:6333 qdrant/qdrant

# Terminal 2: Start FastAPI backend
uvicorn main:app --reload --port 8000

# Terminal 3: Start Streamlit frontend
streamlit run app.py
```

5. Evaluation

```

# evaluate.py
# Use RAGAS to evaluate your RAG system

from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall
)
from datasets import Dataset

# Prepare evaluation data
eval_data = {
    "question": [
        "What is the main contribution of the paper?",
        "What methodology was used?",
        "What are the key findings?"
    ],
    "answer": [
        # Your RAG system's answers
    ],
    "contexts": [
        # Retrieved contexts for each question
    ],
    "ground_truth": [
        # Human-verified correct answers
    ]
}

dataset = Dataset.from_dict(eval_data)
result = evaluate(dataset, metrics=[
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall
])

```

```
print(result)  
# Expected: Each metric > 0.7 for a good RAG system
```

6. Quick Summary

```
Project: AI Research Assistant (PDF Q&A)  
Stack: FastAPI + Streamlit + Qdrant + Sentence Transformers  
Pipeline: Upload PDF → Parse → Chunk → Embed → Store → Query → Retrieve  
→ Generate  
Key Features: Multi-document, citations, metadata filtering  
Evaluation: RAGAS metrics (faithfulness, relevancy, precision, recall)
```

Next Lecture: Latest RAG Research & 2025-2026 Trends — What's new and what's next